



Today's agenda

- ↳ Delete a given node in doubly LL.
- ↳ Add a node before tail in a doubly LL.
- ↳ LRU Cache
- ↳ Clone a LL with random pointer.



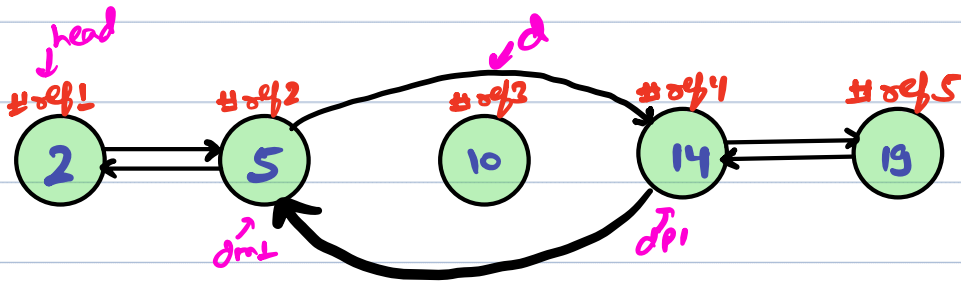
AlgoPrep



Q) Delete a given node from doubly linkedlist.

$d = \text{node 3}$

note: d node can't be head or tail.



```
node delete (Node head, Node d) {
```

```
    Node dml = d.prev;
```

```
    Node dpl = d.next;
```

```
    dml.next = dpl;
```

```
    dpl.prev = dml;
```

```
    d.next = null;
```

```
    d.prev = null;
```

```
    return d;
```

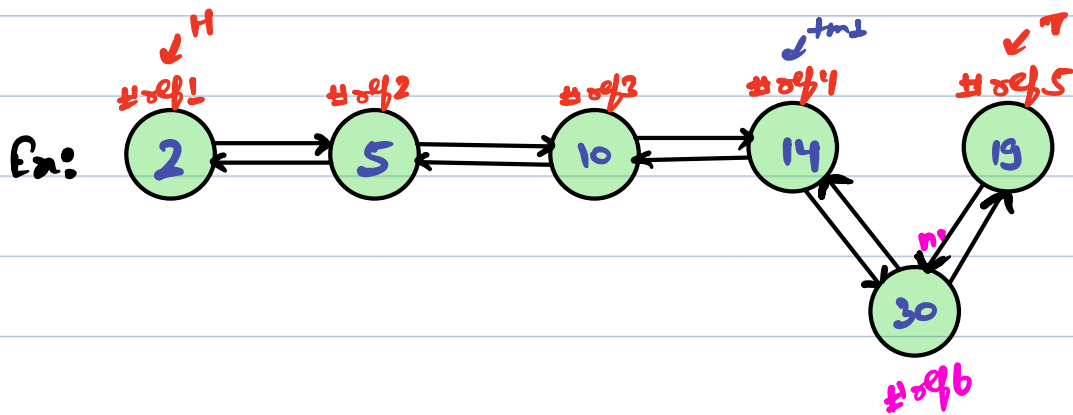
3

T.C: $O(1)$

S.C: $O(1)$



Q) Add a node before tail in a doubly Linked List.



```
void add (Node H, Node T, Node n1) {  
    Node tml = t.prev;
```

```
    tml.next = n1;  
    n1.next = T;
```

T.C: $O(1)$

S.C: $O(1)$

```
    T.prev = n1;  
    n1.prev = tml;
```



Q) LRU Cache → last opened app appears first

↳ Create a least recently used Cache.

facebook 1 Algobep 2 instagoom 3 linkedin 4

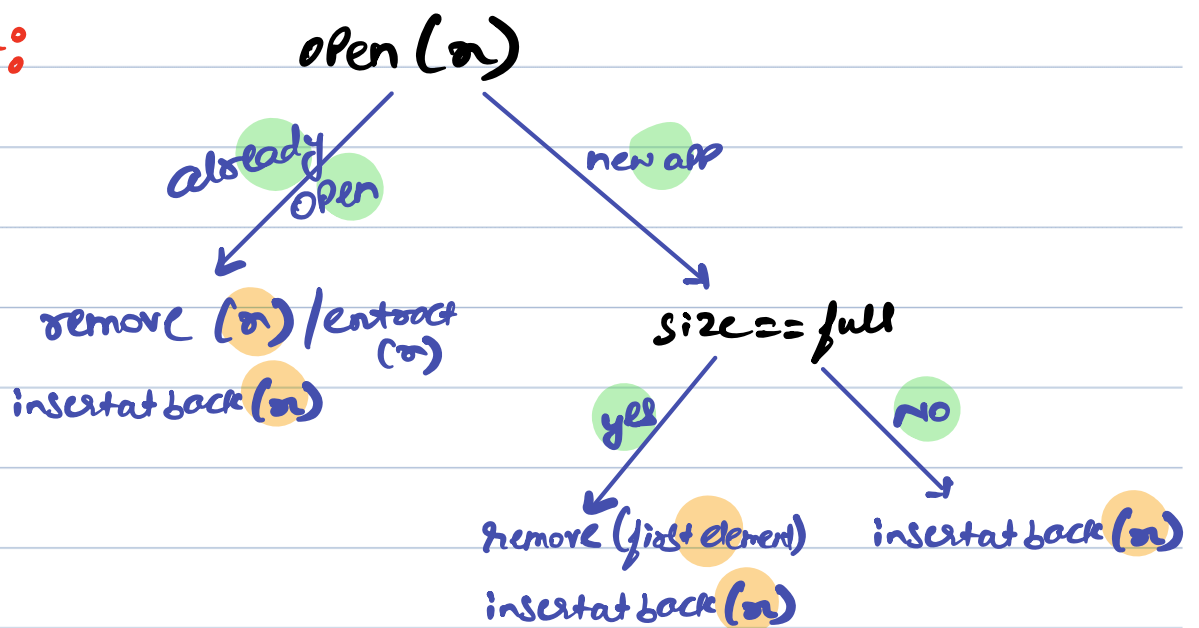
size = 5

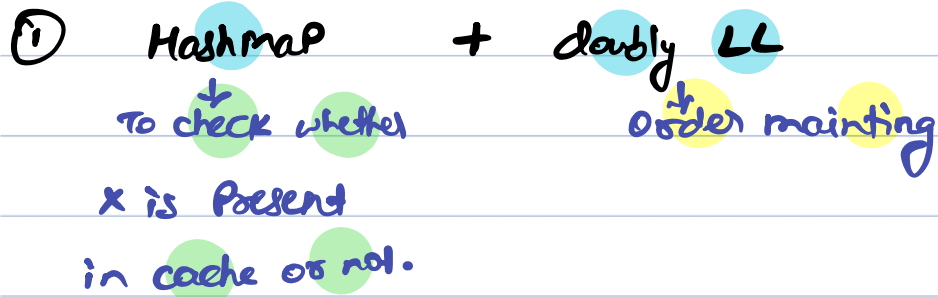
Ex: 7 3 9 2 get() 6 10 get() 14 2

get() 10 get()
↓ ↓
2 10

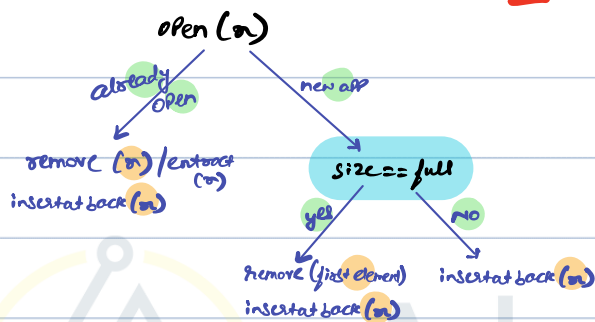
over → recent
[X X 9 X 6 X 14 2 10]

flow chart:





Data: (7, 10) (3, 12) (9, 5) (2, 6) get() 6 get() 2



The diagram shows a linked list with four nodes. The first node is labeled 'old' in pink. The second node contains the value 3 and the address 20. The third node contains the value 9 and the address 50. The fourth node contains the value 6 and the address 10. The fifth node is labeled 'recent' in pink. The list is labeled 'DLL' (Double-Linked List) in blue. A red 'Q' is written below the fourth node.

Integer $\rightarrow$ Node

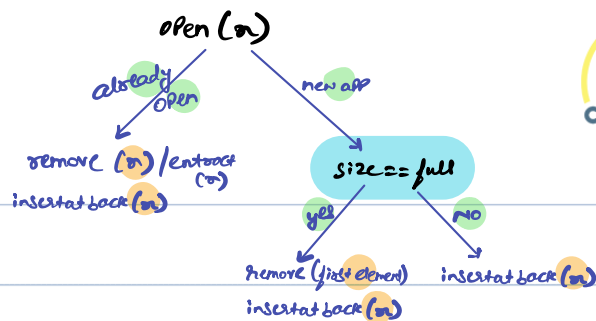
~~7~~ → #2/1

3 → #2/2

9 → #2/3

2 → #2/4

6 → #2/5



//Pseudo code

Class LRUCache {

Class Node {
int key; int val;
Node next; Node prev;

HashMap < Integer, Node > hm;

Node H = new Node();

Node T = new Node();
int cap;

LRUCache (int K) {

cap = K;

hm = new HashMap<>();

H.next = T;

T.prev = H;

}

int get (int key) {

if (hm.containsKey(key) == false) { return -1; }

else {

int ans = hm.get(key).val;

Node temp = delete (H, hm.get(key));

add(H, T, temp);

return ans;

void Put (int key, int value) {

Node res = hm.get(key);

if (res == null) {

if (hm.size() == cap) {

hm.remove(H.next.key);

delete (H.next);

}

Node n1 = new Node()

n1.key = key;

n1.val = value;

hm.put (key, n1);

$O(1) \leftarrow$

$O(1) \leftarrow$

add(n1);



}

else { // res != null

Node temp = remove(H, res);

temp.val = value;

hmap.put(key, temp);

add(temp);

}

}

}

cap = 3

Data: (7, 10) (3, 20) (2, 60) get(3) (6, 100) get(7) (2, 150)

void put(int key, int value) {

Node res = hmap.get(key);

if (res == null) {

if (hmap.size() == cap) {

hmap.remove(H.next.key);

delete(H.next);

}

Node n1 = new Node();

n1.key = key;

n1.val = value;

hmap.put(key, n1);

add(n1);

}

else { // res != null

Node temp = remove(H, res);

temp.val = value;

hmap.put(key, temp);

add(temp);

}

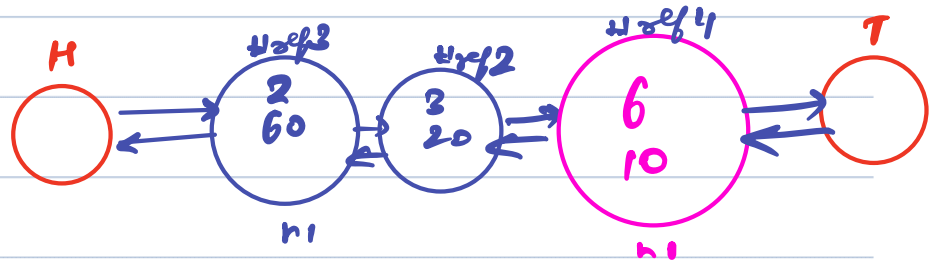
}

7 - #ref 1

3 - #ref 2

2 - #ref 3

6 - #ref 4



int get(int key) {

if (hmap.containsKey(key) == false) { return -1; }

else {

int ans = hmap.get(key).value;

Node temp = delete(H, hmap.get(key));

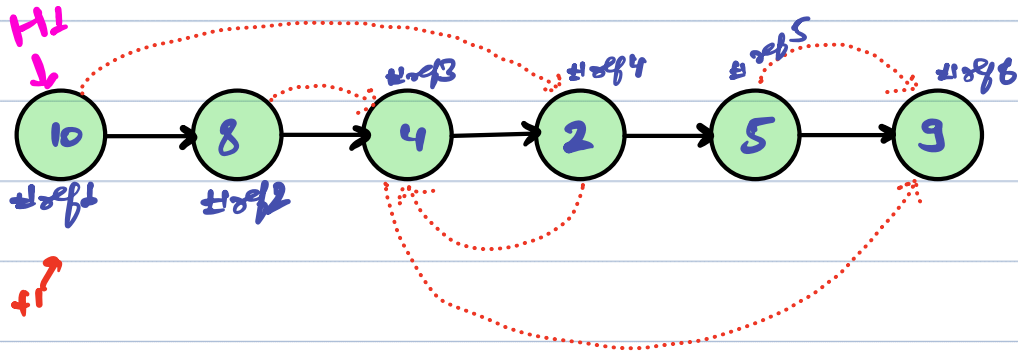
add(H, T, temp);

return ans;

ans = 20

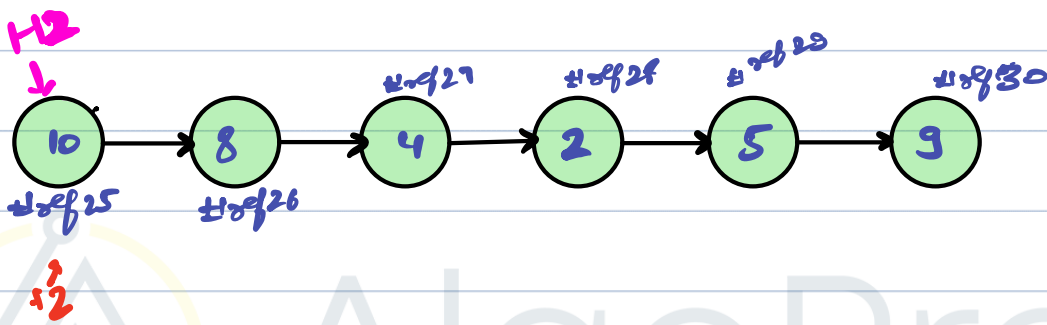


Q) Clone a LL with random pointer.



```
class Node {
    int val;
    Node next;
    Node random;
}
```

3



Idea 1

Hm

Node	Node
#ref1	#ref25
#ref2	#ref26
#ref3	#ref27
#ref4	#ref28

$t2.random = map.get(t1.random);$

$t1 = t1.next$

$t2 = t2.next$

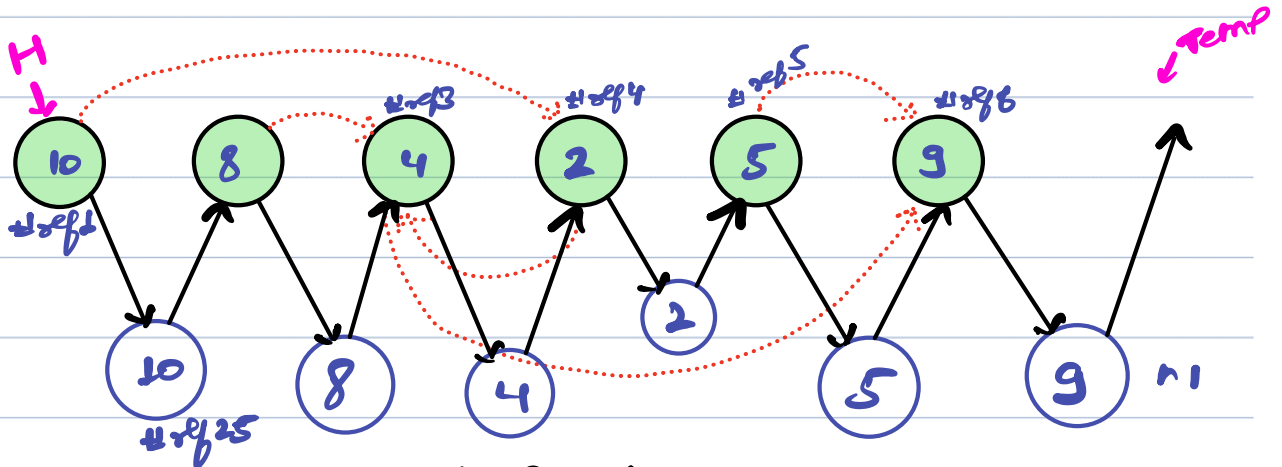
T.C: $O(N)$

S.C: $O(N)$



//idea2

Step 1: insertion



Node temp = H;

while (temp != null) {

Node n1 = new Node(temp.val);

Node tempP1 = n1.next;

temp.next = n1;

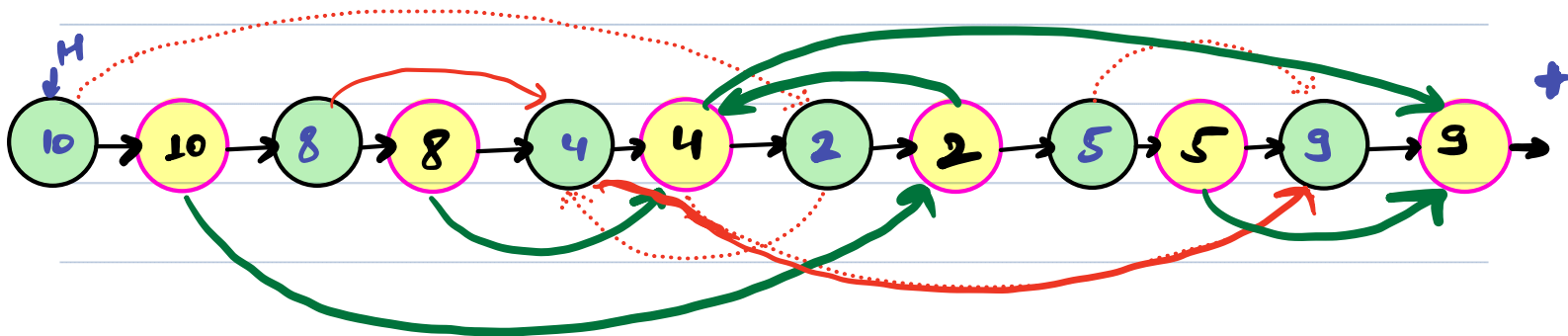
n1.next = tempP1;

temp = tempP1;

}



Step 2: Setting the Random Pointer.



Node $t = H;$

```
while (t != null) {  
    if (t.random != null) {  
        t.next.random = t.random.next;  
    }  
}
```

$t = t.next;$

}

